Facial Emotion Recognition on FER2013 with Frozen CNN Features

CS 413 Affective Computing · Week 7 Lab Report · Track B (deep features + classical classifiers)

Team: [Member 1], [Member 2], [Member 3], [Member 4] · Date: [submission date]
Code and outputs: https://github.com/potakaaa/fer2013-lab

Abstract. *We build a facial emotion recognition pipeline that combines MediaPipe face detection, a frozen ImageNet MobileNetV2 feature extractor and two linear classifiers, and evaluate it on the 7,178-image FER2013 test set. Logistic Regression reaches 49.1% accuracy and 0.451 macro-F1; a Linear SVM reaches 49.7% and 0.448. Happy and surprise are recognised reliably, while disgust, fear and sad remain difficult and are mostly confused with each other and with neutral.*

1 Introduction

Facial emotion recognition (FER) maps a face image to an emotion category and is a core task in affective computing. In this lab we follow Track B: instead of training a network end to end, we use a pretrained convolutional network as a fixed feature extractor and train classical classifiers on its embeddings. This keeps training cheap and shows how far generic ImageNet features carry on a facial-expression task. We compare Logistic Regression with a linear Support Vector Machine (SVM) on FER2013 and wrap the better model in a small prediction function.

2 Dataset

FER2013 (Goodfellow et al., 2013) contains 35,887 grayscale face images of 48×48 pixels, collected through web image search and labelled with one of seven emotions. We use the standard split: the 28,709 *Training* images for training and the 7,178 *PublicTest* and *PrivateTest* images as the test set. The faces are already roughly centred and cropped. Table 1 and Figure 1 show the class distribution. The data is strongly imbalanced: *happy* makes up 25.1% of the training set, while *disgust* has only 436 images (1.5%). We therefore use macro-averaged F1, which weights all seven classes equally, as the main metric.

Table 1. Images per class and split (outputs/class_counts.csv).

Class	Train	Test	Total	Train %
angry	3,995	958	4,953	13.9
disgust	436	111	547	1.5
fear	4,097	1,024	5,121	14.3
happy	7,215	1,774	8,989	25.1
sad	4,830	1,247	6,077	16.8
surprise	3,171	831	4,002	11.0
neutral	4,965	1,233	6,198	17.3
TOTAL	**28,709**	**7,178**	**35,887**	**100.0**

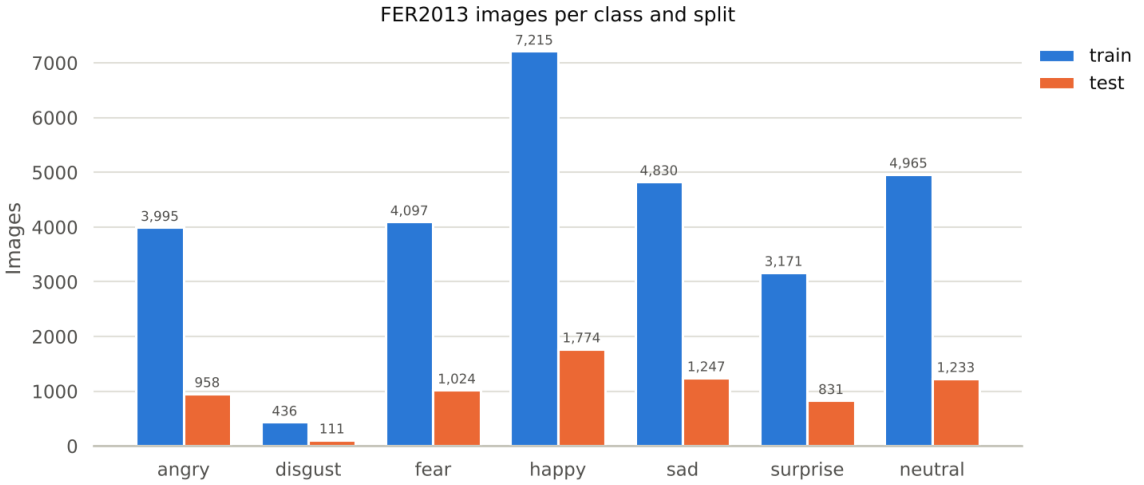


Figure 1. Class distribution per split.



Figure 2. Random training images per class (seed 42).

3 Method

3.1 Face detection (Milestone 1)

Face detectors are trained on photos with some background around the face, so they often fail on FER2013's tiny, tightly cropped images. Each image is therefore upscaled 4× to 192×192 pixels (bicubic) and surrounded by a black border of 25% of its size before detection, so that the face no longer touches the image edge. We use MediaPipe's BlazeFace short-range detector (Bazarevsky et al., 2019) through the Tasks API with a minimum confidence of 0.5; if MediaPipe cannot be loaded, the code falls back to OpenCV's Haar cascade (Viola and Jones, 2001). The reusable function `detect_and_crop(img)` crops the largest detected face with 10% padding; if no face is found it returns the full image and the fallback is counted.

On a seeded sample of 200 training images per class, a face was found in 98.9% of images (Table 2). The border matters: on the same sample the detection rate was 92.6% without it, and wider borders gave no further gain. Figure 3 shows detections and the crops passed to the CNN.

Table 2. Detection rate on a sample of the training set (outputs/m1_detection_rate.csv).

Class	Sampled	Face found	Fallback	Rate %
angry	200	197	3	98.5
disgust	200	194	6	97.0
fear	200	198	2	99.0
happy	200	199	1	99.5
sad	200	196	4	98.0
surprise	200	200	0	100.0
neutral	200	200	0	100.0
TOTAL	**1,400**	**1,384**	**16**	**98.9**



Figure 3. Detected faces (green box) and the resulting crops, three random training images per class.

3.2 Feature extraction (Milestone 2)

Every image goes through the same pipeline: load as grayscale → `detect_and_crop` → resize to 224×224 → replicate to three channels → `mobilenet_v2.preprocess_input` → MobileNetV2. We use MobileNetV2 (Sandler et al., 2018) with ImageNet weights, `include_top=False` and global average pooling, which yields a 1280-dimensional embedding per image. The network is frozen; none of its 2,257,984 parameters are trained. Images are processed in batches of 64, and the embeddings are cached to disk together with the configuration and file list, so re-running the notebook skips this step. Extracting all 35,887 images took 443 s on a 4-core CPU. Table 3 verifies the resulting feature matrices; the label encoding follows the FER2013 convention 0 = angry, 1 = disgust, 2 = fear, 3 = happy, 4 = sad, 5 = surprise, 6 = neutral.

Table 3. Train–test split verification (notebook section 2.3). Train and test files do not overlap.

Split	Images	X shape	y shape	No-face fallback
train	28,709	(28709, 1280)	(28709,)	357 (1.2%)
test	7,178	(7178, 1280)	(7178,)	83 (1.2%)

3.3 Classifiers

Both classifiers are scikit-learn pipelines whose first step is a `StandardScaler`, so the scaling statistics are learned from the training data only. Model 1 is multinomial Logistic Regression (L-BFGS, C = 1, `max_iter=2000`). Model 2 is `LinearSVC` (squared hinge loss, one-vs-rest, C = 1, `max_iter=5000`). We use `LinearSVC` rather than `SVC(kernel='linear')` because kernel SVC scales roughly quadratically with the number of samples, which is impractical for 28,709 training images. All random seeds are fixed to 42.

To handle the class imbalance we compared `class_weight=None` and `'balanced'` (which here weights a disgust sample about 16.5 times more than a happy sample) on a stratified 15% validation split of the training set. The test set was not used for any model choice. Table 4 shows that *balanced* won for both models, but only by a small margin. The chosen configuration was then refitted on the full training set: Logistic Regression converged after 611 of 2000 iterations in 78 s, the SVM after 2,326 of 5,000 iterations in 678 s.

Table 4. Class-weight selection on the validation split (outputs/class_weight_selection.csv).

Model	class_weight	Val. accuracy	Val. macro-F1	Selected
Logistic Regression	None	0.496	0.4479	
Logistic Regression	balanced	0.495	0.4494	✓
Linear SVM	None	0.499	0.4405	
Linear SVM	balanced	0.496	0.4406	✓

4 Results (Milestone 3)

4.1 Overall performance

Table 5 and Figure 4 compare the two models on the test set. They are almost indistinguishable: Logistic Regression is ahead on macro-F1 (0.451 vs 0.448) and macro recall, while the SVM is ahead on accuracy (49.7% vs 49.1%). Because macro-F1 is our selection criterion, **Logistic Regression** is the best model. It also trains about 9 times faster and outputs class probabilities, which the deployment function uses as a confidence score. For context, always predicting *happy* would give 24.7% accuracy on this test set, and human accuracy on FER2013 is about 65% (Goodfellow et al., 2013).

Table 5. Test-set performance (outputs/model_comparison.csv). Bold marks the better model per metric.

Model	Accuracy	Macro precision	Macro recall	Macro-F1	Train time
Logistic Regression	0.491	**0.440**	**0.481**	**0.451**	78 s
Linear SVM	**0.497**	0.438	0.478	0.448	678 s

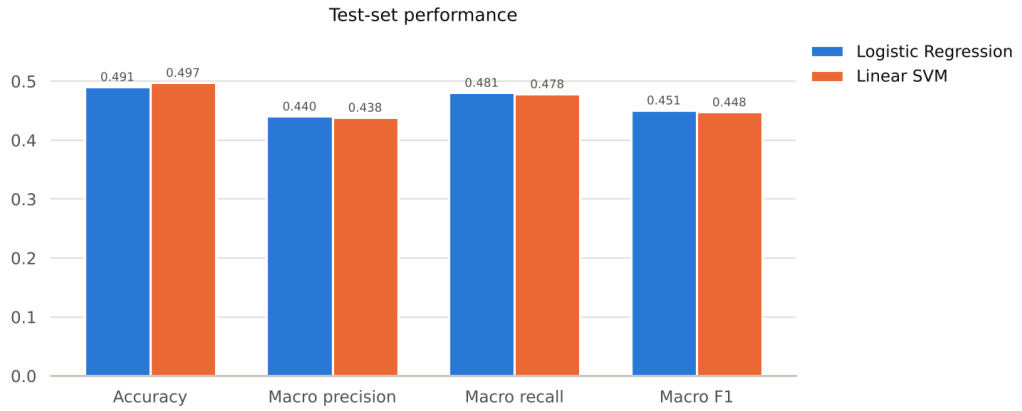


Figure 4. Test-set accuracy and macro-averaged precision, recall and F1.

4.2 Per-class results

Table 6 lists precision, recall and F1 per class. *Happy* (F1 0.689 / 0.695) and *surprise* (0.632 / 0.642) are recognised best: a smile, or a wide-open mouth with raised eyebrows, is a large visual change that survives at 48×48 pixels. *Disgust* (0.280 / 0.268) and *fear* (0.337 / 0.312) are the hardest classes. The per-class F1 scores of the two models differ by at most 0.025. The full classification reports are in notebook sections 4.1 and 4.2.

Table 6. Per-class test metrics (Log. = Logistic Regression, Lin. = Linear SVM; outputs/per_class_metrics.csv).

Class	Log. P	Log. R	Log. F1	Lin. P	Lin. R	Lin. F1	Support
angry	0.371	0.408	0.388	0.381	0.379	0.380	958
disgust	0.199	0.477	0.280	0.192	0.441	0.268	111
fear	0.362	0.315	0.337	0.383	0.263	0.312	1024
happy	0.715	0.665	0.689	0.676	0.716	0.695	1774
sad	0.407	0.342	0.372	0.415	0.343	0.376	1247
surprise	0.581	0.692	0.632	0.570	0.734	0.642	831
neutral	0.449	0.464	0.456	0.452	0.471	0.461	1233

4.3 Confusion matrices

Figure 5 shows the confusion matrix of Logistic Regression as counts and row-normalised (recall); the SVM's matrix (Appendix A) has the same structure. Table 7 lists the five largest off-diagonal entries. Most errors stay within a group of lower-intensity negative expressions: sad is predicted as neutral for 233 images (19% of sad); sad is predicted as angry for 185 images (15% of sad); neutral is predicted as sad for 175 images (14% of neutral); fear is predicted as sad for 173 images (17% of fear). Fear is the most scattered class: its errors spread over sad (17%), angry (14%), surprise (13%) and neutral (13%), and it shares wide eyes and raised brows with surprise.

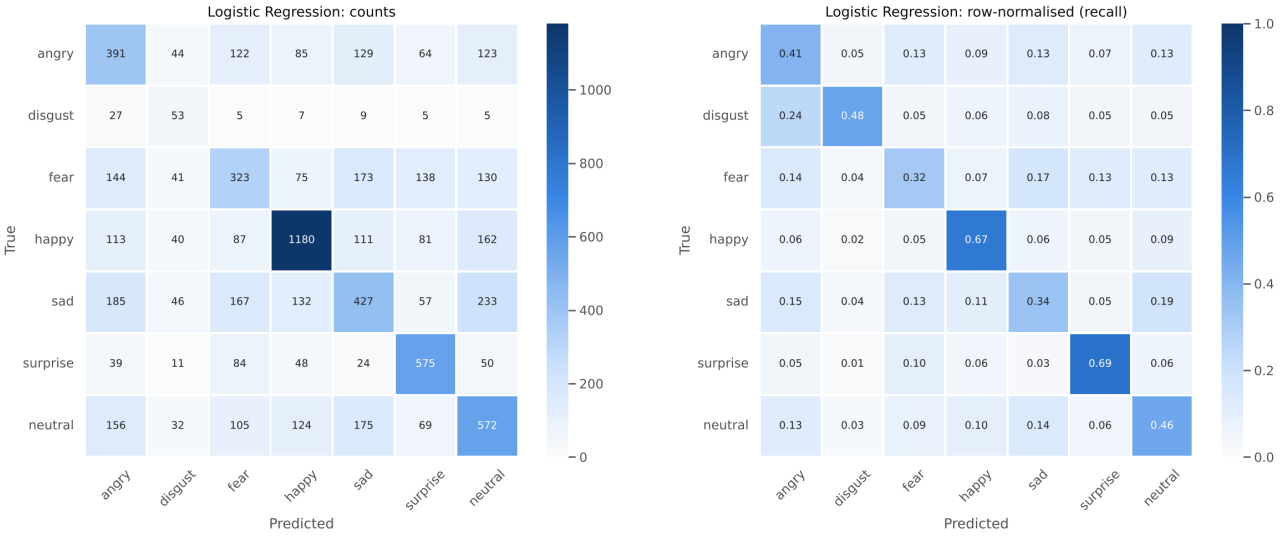


Figure 5. Confusion matrix of Logistic Regression on the test set: counts (left) and row-normalised (right).

Table 7. Most common confusions of Logistic Regression (outputs/top_confusions.csv).

True class	Predicted as	Count	Share of true class
sad	neutral	233	18.7%
sad	angry	185	14.8%
neutral	sad	175	14.2%
fear	sad	173	16.9%
sad	fear	167	13.4%

4.4 Error analysis

Figure 6 shows five misclassified test images, one per true class. Several are hard even for a person: an exaggerated, statue-like grin labelled happy; a fearful face partly covered by a text watermark; an extreme close-up of a child that shows little more than the eyes; a crying infant labelled sad but predicted neutral; and a frown with a wrinkled nose labelled disgust but predicted angry, two cues the classes share. Four of the five predictions have a confidence below 0.5, so the model is visibly unsure on these cases.



Figure 6. Misclassified test images (Logistic Regression): true label, predicted label and confidence.

5 Discussion

5.1 Key insights

- **The two classifiers are effectively tied.** Both are linear models on the same standardised embeddings, so they learn very similar decision boundaries. Logistic Regression is preferred because it wins on macro-F1, trains about nine times faster and produces probabilities.

- **Expression intensity drives difficulty.** Classes with large, distinctive facial changes (happy, surprise) are recognised well; subtle negative expressions (sad, fear, angry) and neutral absorb most of the errors.
- **Disgust is often found but rarely predicted correctly.** With balanced class weights its recall is 0.477, yet only 53 of the 267 disgust predictions are correct (precision 0.199), and 24% of true disgust faces are labelled angry.
- **Class weighting barely matters here.** On the validation split, *balanced* improved macro-F1 by only 0.0015 (Logistic Regression) and 0.0001 (SVM).
- **Face detection is not the bottleneck.** Faces were found in 98.9% of sampled training images and only 1.2% of each split fell back to the full frame, so the remaining error comes from the features and the labels rather than from cropping.

5.2 Limitations

- The features are frozen and come from ImageNet object classification, not faces. The 48×48 images are also upscaled more than four times to 224×224, so the network mostly sees interpolated pixels.
- FER2013 labels are noisy; human accuracy is only about 65% (Goodfellow et al., 2013), which caps what any model can reach on this test set.
- Only linear classifiers were evaluated, on a single train/test split with one seed, so no confidence intervals are reported.
- For the SVM, the confidence is a softmax over decision scores rather than a calibrated probability. This does not affect the deployed model, which is Logistic Regression.

5.3 Possible improvements

- Fine-tune the CNN end to end with data augmentation (flips, small rotations and crops). Fine-tuned networks reach about 73% on FER2013 (Khaireddin and Chen, 2021).
- Train on the FER+ relabelled annotations (Barsoum et al., 2016) to reduce label noise.
- Try a non-linear classifier, such as a small multilayer perceptron, on the same embeddings.

6 Deployment

The function `predict_emotion(image_path) -> (label, confidence)` reuses exactly the functions that produced the training features (`load_gray` → `preprocess_for_cnn` → `embed`) and the saved scikit-learn pipeline (scaler and classifier), loaded from `models/fer_best_model.joblib` (68 KB). Figure 7 shows it on four random test images: two are correct (neutral 0.57, happy 0.92), which is in line with the test accuracy of about 49%. For all four images the prediction matches the batch evaluation, which confirms that deployment and training use identical preprocessing.



Figure 7. predict_emotion() on four unseen test images: file name, predicted label with confidence, and true label.

7 Conclusion

A frozen MobileNetV2 with a linear classifier reaches about 49% accuracy and 0.45 macro-F1 on FER2013, about twice the majority-class baseline but well below human performance. Logistic Regression and a Linear SVM perform almost identically, and Logistic Regression is the better practical choice. Face detection on upscaled images works reliably, so further gains will come from learning face-specific features, for example by fine-tuning the network, rather than from changing the classifier on top of fixed features.

8 Team contributions

Table 8. Team roles (replace the placeholders).

Member	Role	Contributions
[Member 1]	Data & preprocessing lead	Dataset checks, face detection (Milestone 1)
[Member 2]	Feature extraction lead	MobileNetV2 embeddings, feature cache (Milestone 2)
[Member 3]	Modelling & evaluation lead	Training, metrics, error analysis (Milestone 3)
[Member 4]	Deployment & report lead	predict_emotion, demo, report write-up

References

Barsoum, E., Zhang, C., Canton Ferrer, C., and Zhang, Z. (2016). Training deep networks for facial expression recognition with crowd-sourced label distribution. *ACM International Conference on Multimodal Interaction (ICMI)*.

Bazarevsky, V., Kartynnik, Y., Vakunov, A., Raveendran, K., and Grundmann, M. (2019). BlazeFace: Sub-millisecond neural face detection on mobile GPUs. *arXiv:1907.05047*.

Goodfellow, I. J., Erhan, D., Carrier, P. L., Courville, A., Mirza, M., Hamner, B., et al. (2013). Challenges in representation learning: A report on three machine learning contests. *International Conference on Neural Information Processing (ICONIP)*.

Khaireddin, Y., and Chen, Z. (2021). Facial emotion recognition: State of the art performance on FER2013. *arXiv:2105.03588*.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Viola, P., and Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Appendix A Linear SVM confusion matrix

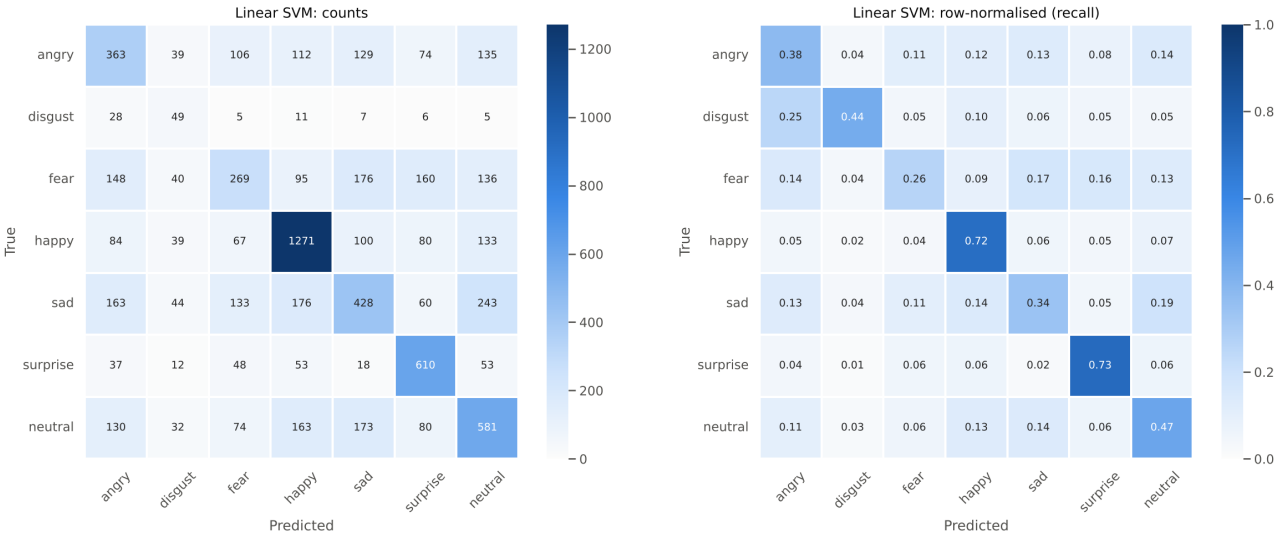


Figure A1. Confusion matrix of the Linear SVM on the test set: counts (left) and row-normalised (right).

Appendix B Reproducibility

All code, figures and tables are in the repository https://github.com/potakaaa/fer2013-lab. The notebook `CS413_Week7_FER_Lab.ipynb` runs top to bottom with *Restart & Run All* (about 36 minutes on a 4-core CPU without a GPU, of which feature extraction takes about 7 minutes and the Linear SVM fits most of the rest). The environment is pinned in `requirements.txt`: Python 3.13, TensorFlow 2.21, MediaPipe 1.1.1, OpenCV 4.14, scikit-learn 1.9.1. The seed is 42 for `random`, NumPy, TensorFlow and every scikit-learn `random_state`; re-extracting the features and retraining reproduces identical metrics. This report is generated from the notebook outputs by `report/build_report.py`.